ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

MULTI-LABEL CODE COMMENT CLASSIFICATION

Traditional Machine Learning and Transformer Models
A Comparative Study on NLBSE'23

Seminar 2 Research Group

Hanoi University of Science and Technology
School of Information & Communication Technology

5 November 2026

ONE LOVE. ONE FUTURE.

Research team and supervision

Team members

Nguyễn Khắc Thái Bình	20251324M
Lã Minh Trung	20251319M
Nguyễn Thu Uyên	20252279M
Nguyễn Thị Nhã Linh	20261262M
Nguyễn Đức Anh	20252469M

Instructor

Dr. Phuong Nguyen
University of L'Aquila, Italy

Course: Seminar 2
Academic year: 2026–2027

Original manuscript: docs/report/report.tex; updated report cover.

One comment can express several information types

Task

Given a comment sentence, predict every applicable information category in its programming-language-specific taxonomy.

Example: “decorated function in the instance itself, used as a context manager.”

Gold labels: Expand *and* Usage (Python, sentence 19).

- Supports comment navigation and documentation analysis.
- Predicting information type does not measure comment quality or correctness.

Report Secs. 1, 3 and 6.7; prepared Python dataset, sentence 19.

Three questions structure the study

1. **RQ1:** How competitive are traditional TF-IDF models against a pretrained Transformer reference?
2. **RQ2:** Which neural configurations perform best across Python, Java and Pharo?
3. **RQ3:** Do improvements in aggregate F1 also improve individual labels?

Five classical learners; seventeen neural configurations; grouped evaluation and per-label analysis.

Report Sec. 1.1.

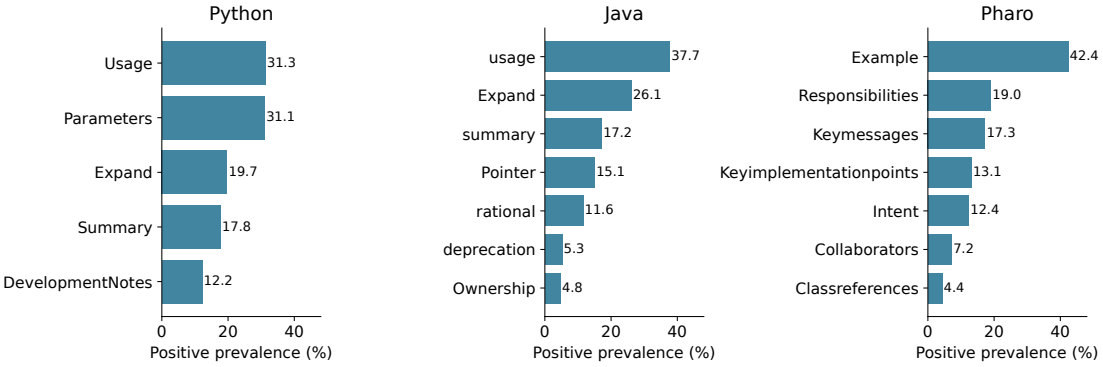
The dataset contains 6,738 comment sentences

Language	Sentences	Labels	Cardinality	Text groups
Python	2,555	5	1.121	2,299
Java	2,418	7	1.177	1,668
Pharo	1,765	7	1.158	1,620

- Sentence IDs join category-wise records into binary target vectors.
- Taxonomies differ by language: 19 language-specific categories in total.
- Exact whitespace-normalized duplicate texts remain in one group.

Report Sec. 3, Tables 2–3; NLBSE dataset: github.com/nlbse2023/code-comment-classification.

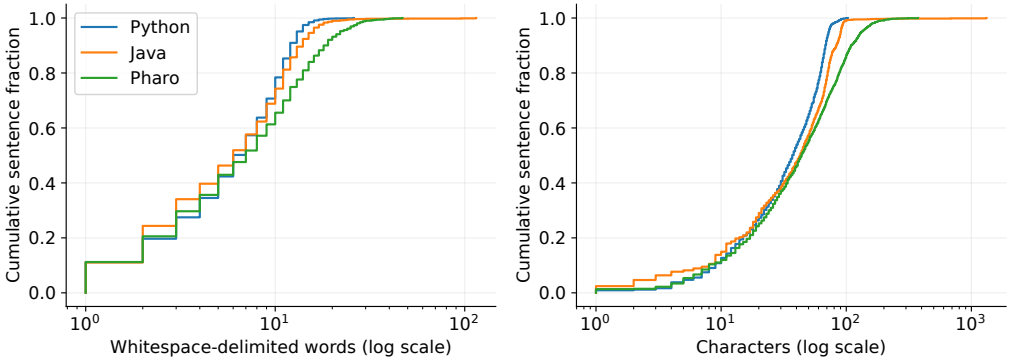
Label support is strongly imbalanced



Usage dominates Python; usage dominates Java; Example dominates Pharo.

Report Fig. 1; computed from full prepared targets (descriptive EDA).

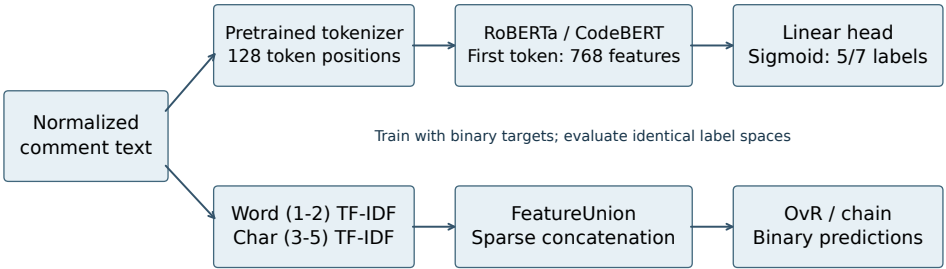
Most sentences are short, but tails differ



Median whitespace-delimited words: Python 6, Java 6, Pharo 7.
These are text statistics, not tokenizer subword counts.

Report Fig. 2; prepared sentence texts.

Models share targets and evaluation interfaces



TF-IDF models learn sparse lexical patterns; pretrained encoders learn contextual representations.

Report Fig. 6; src/models/ml and src/models/dl.

Five classical models use fixed settings

Method	Main recorded settings
Logistic regression	One-vs-Rest; $C = 5$; balanced weights
Classifier chain	Logistic base; random label order; seed 42
Linear SVM	One-vs-Rest; $C = 1$; balanced weights
SGD	Logistic loss; balanced weights; max. 2,000 iterations
Random forest	One-vs-Rest; 300 trees; balanced weights

Word 1–2 grams + character 3–5 grams; fold-local TF-IDF fitting. No systematic hyperparameter search is documented for these archived runs.

Report Sec. 5.1, Table 5; fixed settings are not an equal tuning budget to neural screening.

Seventeen neural configurations test several factors

Factor	Configuration examples
Backbone	E1: RoBERTa BCE; E11: CodeBERT BCE
Loss	E2: weighted BCE; E3/E17: focal loss; E7: ASL
Label structure	E5/E6: label attention; E12: label GNN
Regularization	E8: R-Drop; E10: adversarial; E15: CodeBERT R-Drop
Optimization	E9: patience/tie-break policy; E13/E14: layer-wise learning rates

E16 combines interventions. E4 shares E1 screening scores because threshold tuning is deferred; its final run is absent.

Report Sec. 5.2, Table 6; complete E1–E17 settings remain in report.

Multi-label objectives act on separate label scores

For each label ℓ, the encoder predicts $p_\ell = \sigma(z_\ell)$.

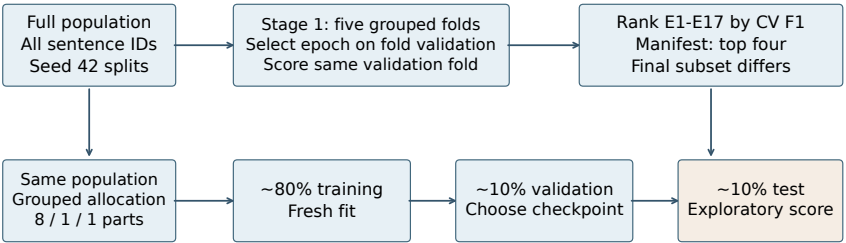
$$\mathcal{L}_{\text{BCE}} = -\frac{1}{L}\sum_{\ell=1}^{L}[y_\ell \log p_\ell + (1 - y_\ell)\log(1 - p_\ell)].$$

- **Focal loss:** reduces the contribution of easy decisions.
- **ASL:** uses asymmetric focusing for positive and negative labels.
- **R-Drop:** penalizes disagreement between stochastic forward passes.

Reported final neural decisions use threshold 0.5.

Report Sec. 5.3; Lin et al. (2017), Ridnik et al. (2021), Wu et al. (2021); bibliography in report.

Grouped screening precedes an exploratory final split



Final test sentences already participated in Stage 1: no independent holdout or nested CV

Stage 1: grouped five-fold CV. Stage 2: approximately 80/10/10.
Final test sentences already participated in Stage 1 method selection.

Report Sec. 5.4, Fig. 7. Final testing is not independent confirmation.

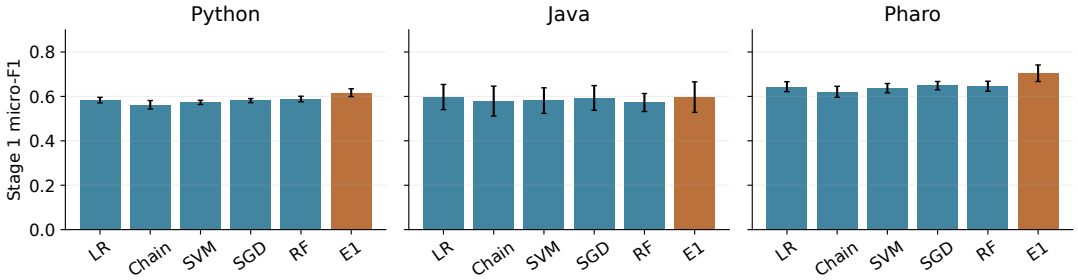
Micro-F1 and label-level scores answer different questions

$$F1_{\text{micro}} = \frac{2\sum_{\ell} TP_{\ell}}{2\sum_{\ell} TP_{\ell} + \sum_{\ell} FP_{\ell} + \sum_{\ell} FN_{\ell}}, \qquad F1_{\text{macro}} = \frac{1}{L}\sum_{\ell} F1_{\ell}.$$

- **Micro-F1:** pools decisions; frequent labels have greater influence.
- **Macro-F1:** weights labels equally; small supports can be unstable.
- **Precision, recall, support:** explain minority-label trade-offs.
- Hamming loss and average precision provide complementary views.

Report Sec. 5.5; no statistical significance inferred from fold SD.

RQ1: classical models remain competitive on Java



Best classical vs. E1: Python 0.5882 vs. 0.6169; Java 0.5972 vs. 0.5968; Pharo 0.6486 vs. 0.7044. Error bars show fold SD, not confidence intervals.

Report Sec. 6.1, Fig. 8; Stage 1 mean micro-F1.

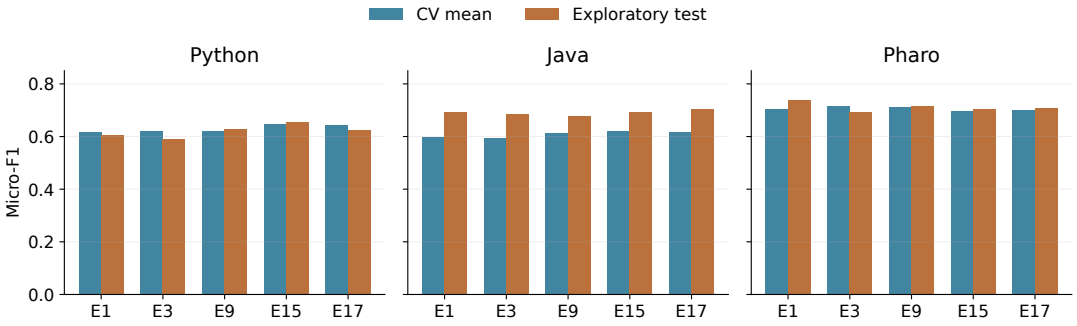
RQ2: the strongest neural choice depends on language

Configuration	Python	Java	Pharo
E1: RoBERTa BCE	0.6169	0.5968	0.7044
E3: RoBERTa focal	0.6219	0.5954	**0.7152**
E11: CodeBERT BCE	0.6397	0.5980	0.6805
E15: CodeBERT R-Drop	**0.6479**	**0.6225**	0.6956

- E15 leads Python and Java screening; E3 leads Pharo.
- Code pretraining alone does not guarantee an improvement (E11 vs. E1).
- E3–E1 on Pharo: +1.08 pp, from displayed four-decimal scores.

Report Sec. 6.2, Table 8; descriptive Stage 1 means, not significance-tested effects.

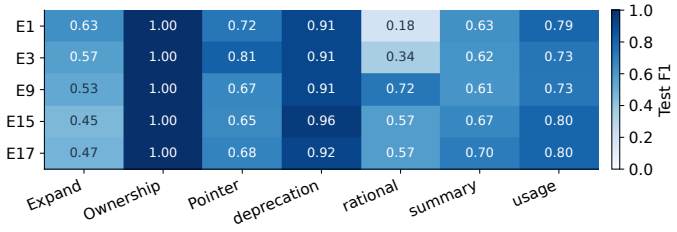
Final results are available for a fixed neural subset



Highest observed final micro-F1: Python E15 0.6541; Java E17 0.7034; Pharo E1 0.7371.
Final set: E1/E3/E9/E15/E17; different fits and scoring sets from CV.

Report Secs. 6.3–6.4, Fig. 9. Bars must not be read as generalization-gap estimates.

RQ3: aggregate gains can hide label-level regressions



Java: E1 → E17

- Micro-F1: 0.6947 → 0.7034.
- rational F1: 0.182 → 0.571.
- Expand F1: 0.630 → 0.474.
- Ownership F1: 1.000 for both; only 11 positives.

Report Sec. 6.5, Table 13 and Fig. 11; retrospective descriptive comparison.

A difficult label improves through higher recall

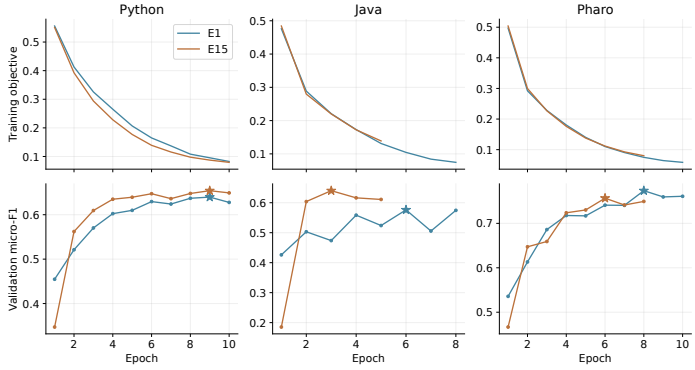
Java rational: 28 positive test examples

Model	TP	FP	FN	Recall	F1
E1	3	2	25	0.1071	0.1818
E17	12	2	16	0.4286	0.5714

- Nine additional true positives; false positives remain at two.
- Aggregate counts support this explanation.
- Missing row-level predictions prevent identifying which sentences changed.

Report Sec. 6.7; stored Java final per-label metrics (E1 and E17).

Validation stopping can precede the lowest training loss



Java E15 peaks at epoch 3 (validation micro-F1 0.6402), then declines while training loss falls. Stars mark selected epochs; objective magnitudes differ across E1 and E15.

Report Sec. 6.6, Fig. 14; stored training histories, not newly trained models.

Published benchmarks provide context, not a direct ranking

Official method	Avg. precision	Avg. recall	Avg. F1
Random forest baseline	0.439	0.245	0.309
Submitted logistic regression	0.540	0.560	0.547
CodeT5	0.728	0.606	0.657
STACC	0.710	0.794	0.744

The official benchmark averages 19 binary-category scores on official partitions. Our per-language micro-F1 and grouped protocol are different. No superiority margin is claimed.

Kallis et al., NLBSE tool competition overview (2023), Table III; ojcchar.github.io/files/24-nlbse23-tools.pdf.

The tuning budgets are unequal

Classical family

- Five learned methods.
- Fixed hyperparameter settings.
- No systematic search documented in archived results.

Transformer family

- Seventeen configurations screened.
- Multiple architecture/loss/optimization choices.
- Validation-based checkpoint selection.

Interpretation

These are results for recorded configurations, not equally tuned model families. A controlled follow-up needs comparable search budgets and an independent outer test set.

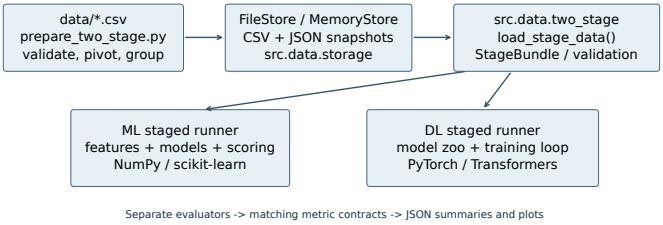
Report Sec. 8.1; archived configuration coverage.

Evaluation limitations bound the conclusions

- **Selection dependence:** Stage 1 includes future final-test sentences.
- **Checkpoint selection:** the same CV fold chooses the epoch and supplies its reported score.
- **Limited replication:** one base seed; fold SD is not seed uncertainty or a confidence interval.
- **External validity:** no project-level holdout; taxonomies and label supports differ.
- **Missing artifacts:** original row-level predictions and distributed weights are unavailable.

Report Sec. 8. No new statistical significance or unseen-project claim.

A shared pipeline supports reproducibility



- Shared validated targets, folds and fingerprints.
- FileStore / MemoryStore for staged dataset persistence.
- 35 offline software tests pass.
- Six prepared datasets regenerate identically.
- Plots and model exports remain filesystem-based.

Report Secs. 4 and 9; docs/storage.md. Adapters are revision-time additions; archived training was not rerun.

Conclusions and the next controlled experiment

1. **RQ1:** lexical baselines remain useful; Java screening performance is competitive with E1.
2. **RQ2:** the best recorded neural configuration varies by language and evaluation subset.
3. **RQ3:** aggregate gains require per-label precision, recall and support analysis.

Next experiment

Freeze an independent project-grouped test set; equalize tuning budgets; repeat seeds; export row-level predictions and checkpoints.

Report Secs. 7–10. Results are descriptive within the recorded protocol.

Selected references

1. Kallis et al. (2023). The NLBSE'23 tool competition.
2. Pascarella & Bacchelli (2017). Classifying code comments in Java open-source software.
3. Liu et al. (2019). RoBERTa.
4. Feng et al. (2020). CodeBERT.
5. Lin et al. (2017). Focal loss for dense object detection.
6. Wu et al. (2021). R-Drop.
7. Al-Kaswan et al. (2023). STACC.

Full 19-entry bibliography and primary-source links: report_rebuilt/references.bib. Theme: github.com/k1m1nonawa4869/HUST-Beamer-Theme.